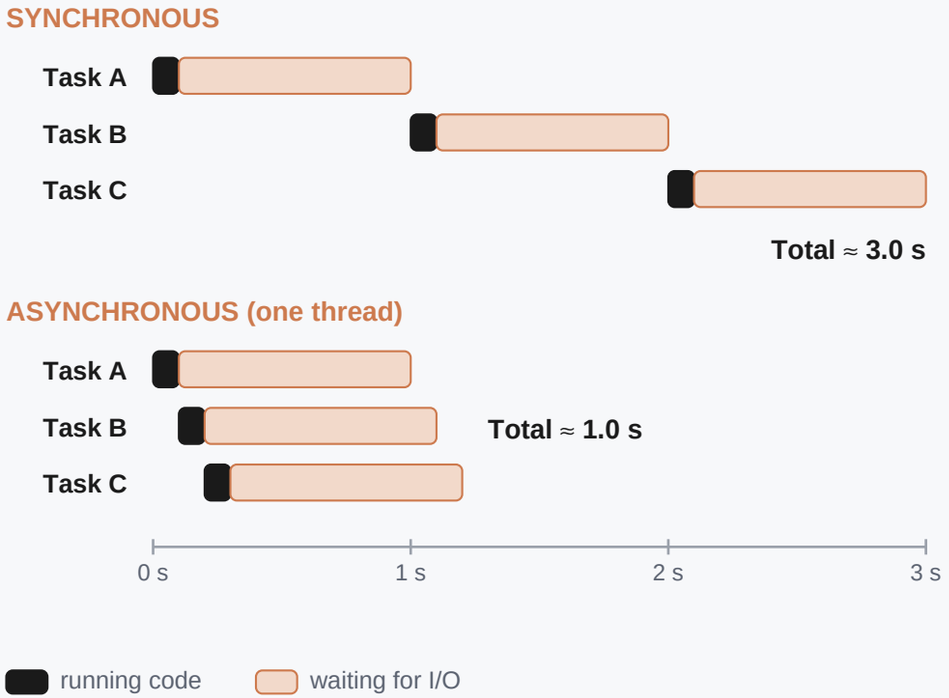
Developing Concurrent & Asynchronous Applications

Ensuring High Performance and Responsive Applications

FOUNDATIONS

Synchronous vs. Asynchronous Execution

- Synchronous: each task must finish before the next one starts. While a task waits for I/O, the whole program sits idle.
- Asynchronous: when a task starts waiting, the program switches to another task and comes back when the wait is over.
- Both versions do exactly the same work; the asynchronous version simply overlaps the waiting time.
- The gain is largest for I/O-bound work: network calls, database queries and file access.



NOTE

Three tasks that each wait 1 second take about 3 seconds when run synchronously, but about 1 second when run asynchronously, because the three waits overlap. The next slide shows the same idea in code.

FOUNDATIONS

Seeing the Difference in Code

The same three 1-second tasks, written synchronously (left) and asynchronously (right).

Synchronous: time.sleep() blocks

```python
import time

def fetch(name):
    print(f"start {name}")
    time.sleep(1)      # blocks the program
    print(f"done {name}")

def main():
    for name in ["A", "B", "C"]:
        fetch(name)

start = time.perf_counter()
main()
print(f"{time.perf_counter() - start:.1f} s")
# start A, done A, start B, done B,
# start C, done C, 3.0 s
```

Asynchronous: await yields control

```python
import asyncio, time

async def fetch(name):
    print(f"start {name}")
    await asyncio.sleep(1)  # yields control
    print(f"done {name}")

async def main():
    await asyncio.gather(
        fetch("A"), fetch("B"), fetch("C"))

start = time.perf_counter()
asyncio.run(main())
print(f"{time.perf_counter() - start:.1f} s")
# start A, start B, start C,
# done A, done B, done C, 1.0 s
```

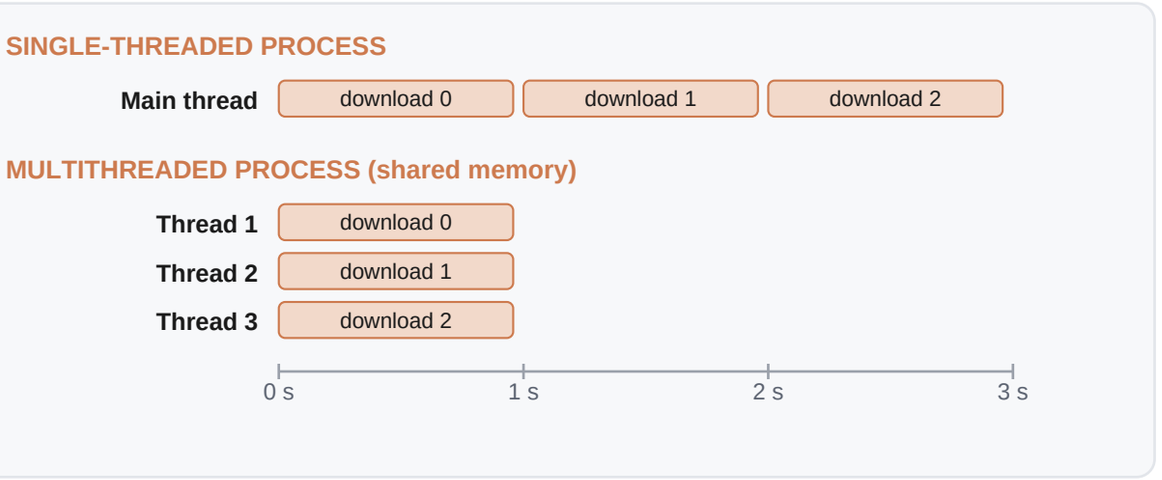
NOTE

time.sleep() freezes the whole program, so the tasks run one after another. await asyncio.sleep() pauses only the current coroutine, so the event loop starts the other tasks while it waits: same work, one third of the time.

FOUNDATIONS

Single-Threaded vs. Multithreaded Applications

- A thread is one sequence of instructions. Every program starts with one: the main thread.
- Single-threaded: one task at a time, so a slow task delays everything behind it.
- Multithreaded: several threads inside one process share the same memory and take turns on the CPU.




```python
import threading, time

def download(n):
    time.sleep(1)          # waiting on I/O
    print(f"file {n} done")

# Single-threaded: about 3 seconds
start = time.perf_counter()
for n in range(3):
    download(n)
print(f"{time.perf_counter() - start:.1f} s")

# Multithreaded: about 1 second
start = time.perf_counter()
threads = [threading.Thread(target=download,
                            args=(n,))
           for n in range(3)]
for t in threads:
    t.start()
for t in threads:
    t.join()
print(f"{time.perf_counter() - start:.1f} s")
```

NOTE

Threads overlap their waiting time, so three 1-second downloads finish in about 1 second instead of 3. Whether threads can also compute in parallel depends on the Global Interpreter Lock, explained next.

FOUNDATIONS

The Global Interpreter Lock (GIL)

- The GIL is a lock inside the standard CPython interpreter: only the thread holding it may run Python bytecode.
- Its role is to keep the interpreter's own memory management (such as reference counting) safe without locking every object.
- A thread releases the GIL while it waits for I/O, so threads still speed up I/O-bound programs.
- CPU-bound work gains little from threads. Use multiprocessing instead: each process has its own interpreter and GIL.

Three threads sharing one GIL




```python
import sys
# Python 3.13 or later
print(sys._is_gil_enabled())
# True  on the standard build (GIL on)
# False on a free-threaded build,
#       for example python3.15t
```

NOTE

Removing the GIL: Python 3.13 introduced an experimental free-threaded build (PEP 703), and Python 3.14 made it officially supported but optional (PEP 779). Python 3.15, the version now in final pre-release development (release candidate 3; final release scheduled for 9 October 2026), continues this work. The standard interpreter still has the GIL; the free-threaded build is installed separately (python3.15t).

MULTITHREADING

Multithreading in Python

- threading.Thread creates a real operating-system thread that shares the program’s memory.
- start() begins running the target function; join() makes the caller wait until it finishes.
- The operating system decides when to switch between threads; the programmer does not control this.
- Because of the GIL, threads suit I/O-bound work (network, files) rather than heavy computation.

```python
import threading
import time

def worker():
    print("Thread started")
    time.sleep(0.5)
    print("Thread done")

def main():
    # Create thread object
    t = threading.Thread(target=worker)

    # Start the thread execution
    t.start()

    # Block main thread until t completes
    t.join()
    print("Main finished.")

if __name__ == "__main__":
    main()
```

NOTE

The target function runs in an independent OS thread. Calling start() schedules it, and join() blocks the main thread until it completes, ensuring an orderly finish.

The Main Thread vs. The Target Thread

The Main Thread

- Starts automatically when you run a Python script.
- Runs your code from top to bottom.
- Creates and manages the other threads.
- Handles final shutdown and clean-up.

The Target Thread

- Created with threading.Thread(target=my_function).
- Runs independently alongside the main thread.
- Executes only the code in its target function.
- Ends automatically when that function returns.



NOTE

Every Python program has exactly one main thread, created automatically at start-up. Target threads are created by the developer and exist only for the lifetime of the function they run.

MULTITHREADING

Race Conditions and Thread Safety

- Threads share memory, so two threads can read and write the same variable at the same moment.
- counter += 1 is really three steps (read, add, write); a switch between them loses an update.
- This is a race condition: results vary from run to run, which makes the bug hard to reproduce.
- Protect every shared read-and-write section with a Lock (next slide).

```python
import threading
import time

counter = 0

def increment():
    global counter
    for _ in range(100000):
        temp = counter        # Read
        time.sleep(0)         # Force a context switch here
        counter = temp + 1    # Write

def main():
    threads = [
        threading.Thread(target=increment)
        for _ in range(2)
    ]
    for t in threads: t.start()
    for t in threads: t.join()
    # Expected: 200000
    # Actual: reliably less
    print(f"Counter: {counter}")

if __name__ == "__main__":
    main()
```

NOTE

Two threads update an unprotected counter. The forced context switch (time.sleep(0)) between the read and the write makes the lost updates happen on every run rather than by chance.

MULTITHREADING

Synchronising Threads with a Lock

- threading.Lock() lets only one thread at a time into a protected (critical) section.
- with lock: acquires the lock on entry and always releases it on exit, even after an error.
- The read and the write now happen together, so the other thread cannot interrupt halfway.
- The forced switch with time.sleep(0) is still there, but the result is now always correct.

```python
import threading
import time

counter = 0
lock = threading.Lock()

def increment():
    global counter
    for _ in range(100000):
        with lock:
            temp = counter      # Read
            time.sleep(0)       # Still forces a switch...
            counter = temp + 1  # ...but the lock keeps
                                # the other thread out

def main():
    threads = [
        threading.Thread(target=increment)
        for _ in range(2)
    ]
    for t in threads: t.start()
    for t in threads: t.join()
    # Expected: 200000
    # Actual: always 200000
    print(f"Counter: {counter}")

if __name__ == "__main__":
    main()
```

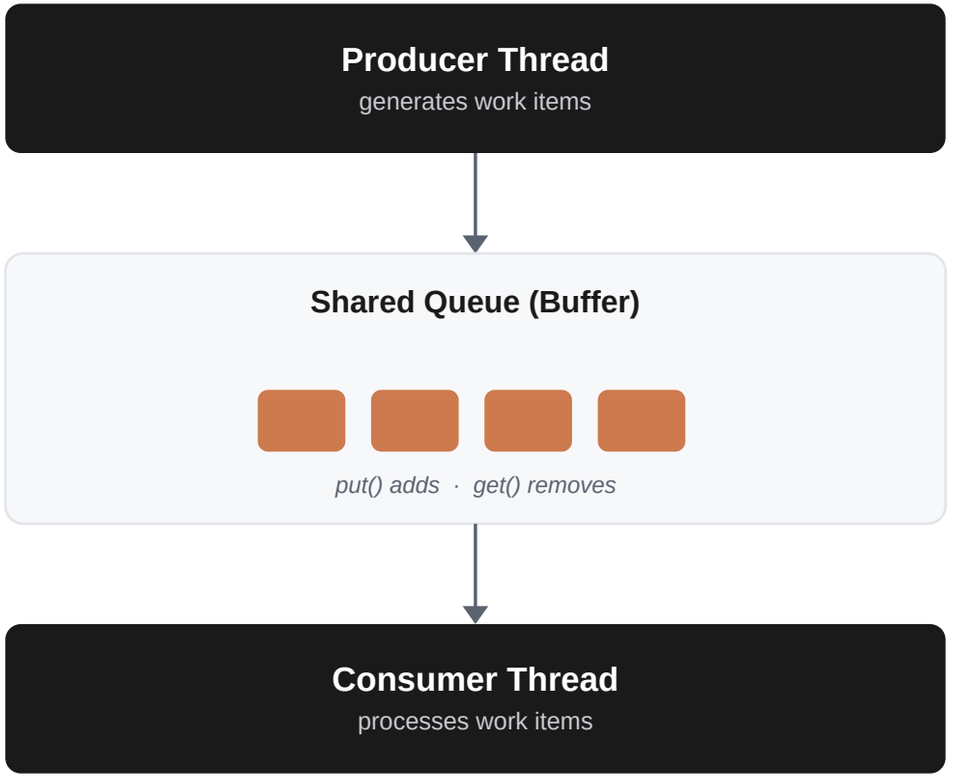
NOTE

The lock turns read-modify-write into a single step from the other thread's point of view, so the count is correct on every run.

MULTITHREADING

What Is the Producer-Consumer Pattern?

- Producer threads create work items; consumer threads process them.
- A shared buffer between them lets each side work at its own speed.
- Unprotected access to that buffer would cause the race conditions seen earlier.
- A thread-safe queue contains its own lock, so no manual locking is needed.
- Common in web servers, logging pipelines and background job systems.



NOTE

The queue on the next slide does the same job as the Lock on the previous slide, packaged for safely handing items from one thread to another.

MULTITHREADING

Producer-Consumer with Threads

- queue.Queue(maxsize=3) is a bounded, thread-safe buffer.
- put() waits when the queue is full; get() waits when it is empty, so no busy-waiting is needed.
- A sentinel value (None) tells the consumer that no more items are coming.
- The consumer checks for the sentinel and leaves its loop cleanly.

```python
import queue, threading, time

q = queue.Queue(maxsize=3)

def producer():
    for i in range(5):
        print(f"Producing: {i}")
        q.put(i)           # Blocks if full
        time.sleep(0.1)
    q.put(None)            # Sentinel

def consumer():
    while True:
        item = q.get()     # Blocks if empty
        if item is None:
            break
        print(f"Consumed: {item}")
        time.sleep(0.2)

def main():
    p = threading.Thread(target=producer)
    c = threading.Thread(target=consumer)
    p.start(); c.start()
    p.join(); c.join()

if __name__ == "__main__":
    main()
```

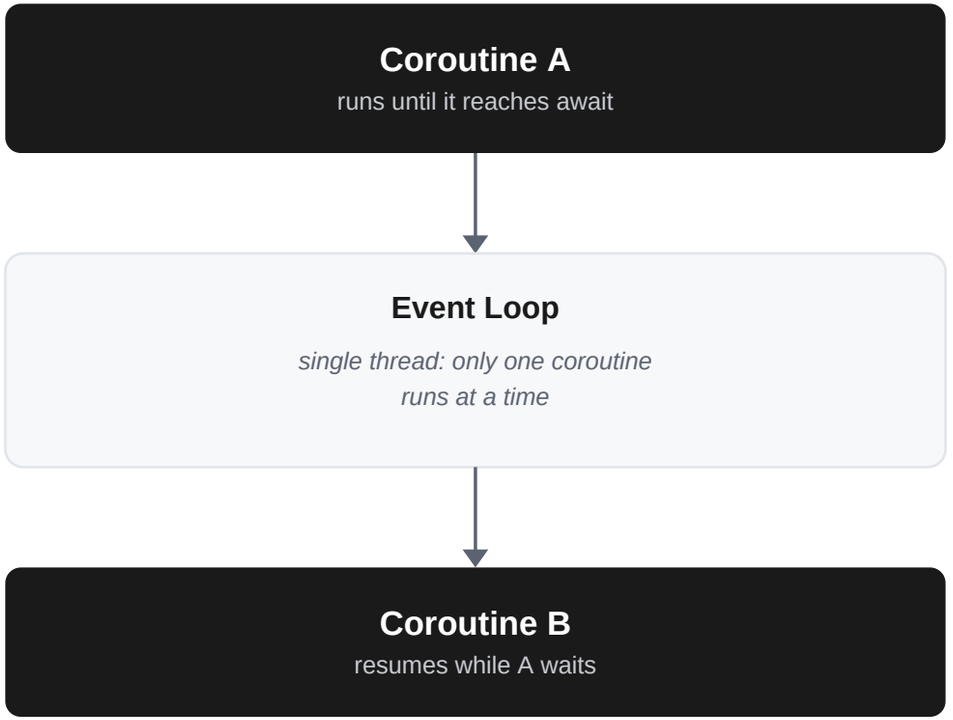
NOTE

The producer and consumer run at different speeds; the bounded queue keeps them in step, and the None sentinel signals a clean shutdown.

ASYNCIO BASICS

Key asyncio Terms vs. Multithreading

- Coroutine: a function defined with async def that can pause and later resume where it stopped.
- await: the point where a coroutine pauses and hands control back to the event loop.
- Event loop: a single-threaded scheduler that runs one coroutine at a time.
- Threads are switched by the OS at almost any point (pre-emptive); coroutines switch only at await (cooperative).
- Fewer switch points mean fewer surprise race conditions, although shared state used across an await may still need an asyncio.Lock (covered later).



NOTE

Multithreading: many threads, and the operating system decides when to switch. asyncio: one thread, and the code decides when to switch, only at await.

ASYNCIO BASICS

Introduction to asyncio and Coroutines

- asyncio runs many tasks on one thread using cooperative multitasking.
- A coroutine gives control back to the event loop at each await.
- Coroutines are far lighter than OS threads, so thousands can run at once.
- asyncio.run() creates the event loop, runs the main coroutine and closes the loop.

```python
import asyncio

async def greet():
    print("Hello...")
    # Yield control to the loop for 0.5 s
    await asyncio.sleep(0.5)
    print("...World!")

async def main():
    # Awaiting suspends main until greet is done
    await greet()

if __name__ == "__main__":
    # Creates, runs and closes the event loop
    asyncio.run(main())
```

NOTE

await asyncio.sleep() hands control back to the event loop, which could run other tasks during the pause. Here there is only one task, so the next slides add more.

ASYNC CONCURRENCY

Concurrency with asyncio.create_task

- create_task() wraps a coroutine in a Task and schedules it to start straight away.
- Several tasks can then make progress concurrently on the same single thread.
- Awaiting a task later simply collects its result once it has finished.
- Total time is roughly that of the longest task, not the sum of all tasks.

```python
import asyncio

async def fetch(id_code):
    print(f"Start fetch {id_code}...")
    await asyncio.sleep(0.5)
    print(f"End fetch {id_code}")
    return f"Data {id_code}"

async def main():
    # Both tasks start running immediately
    t1 = asyncio.create_task(fetch("A"))
    t2 = asyncio.create_task(fetch("B"))

    # Collect the results
    res1 = await t1
    res2 = await t2
    print(f"Results: {res1}, {res2}")

if __name__ == "__main__":
    asyncio.run(main())
```

NOTE

Both tasks start before either is awaited, so their waits overlap and the total time falls from about 1.0 second to about 0.5 seconds.

ASYNC CONCURRENCY

Orchestrating with asyncio.gather

- asyncio.gather() runs several awaitables concurrently and waits until all of them have finished.
- Results come back in the order the awaitables were passed in, not the order they finish.
- Ideal for a batch of similar operations, such as many API calls or web pages.
- Removes the need to create and await each task by hand.

```python
import asyncio

async def process(val):
    await asyncio.sleep(0.1)
    return val * 10

async def main():
    # Run several coroutines concurrently
    results = await asyncio.gather(
        process(1),
        process(2),
        process(3)
    )
    # Returned in call order
    print(f"Gathered results: {results}")

if __name__ == "__main__":
    asyncio.run(main())
```

NOTE

gather runs the coroutines concurrently on a single thread and returns their results in the original call order: [10, 20, 30].

ASYNC CONCURRENCY

Resource Synchronisation with asyncio.Lock

- Only one coroutine runs at a time, but an await in the middle of an update lets another coroutine run.
- asyncio.Lock lets only one coroutine at a time enter the protected section.
- async with lock: releases the lock automatically, even if an error occurs.
- Coroutines waiting for the lock queue up and resume in turn.

```python
import asyncio

counter = 0
lock = asyncio.Lock()

async def increment():
    global counter
    async with lock:
        val = counter
        # await yields, but the lock blocks others
        await asyncio.sleep(0.01)
        counter = val + 1

async def main():
    await asyncio.gather(
        *(increment() for _ in range(5))
    )
    print(f"Counter: {counter}")   # Counter: 5

if __name__ == "__main__":
    asyncio.run(main())
```

NOTE

Only one task at a time can run the read, await, write sequence, so the counter always ends at 5.

ASYNC CONCURRENCY

The asyncio Producer-Consumer Model

- asyncio.Queue passes data between coroutines.
- await q.put() pauses when the queue is full; await q.get() pauses when it is empty.
- maxsize limits memory use by slowing the producer when the consumer falls behind.
- task_done() and q.join() let the program wait until every item has been processed.
- The consumer loops forever, so it is cancelled once the queue is empty.

```python
import asyncio

q = asyncio.Queue(maxsize=3)

async def producer():
    for i in range(5):
        await q.put(i)          # Suspends if full
        print(f"Produced: {i}")
        await asyncio.sleep(0.1)

async def consumer():
    while True:
        item = await q.get()    # Suspends if empty
        print(f"Consumed: {item}")
        q.task_done()
        await asyncio.sleep(0.2)

async def main():
    c_task = asyncio.create_task(consumer())
    await producer()
    await q.join()              # Wait until all processed
    c_task.cancel()
    try:
        await c_task
    except asyncio.CancelledError:
        pass
    print("All items processed.")

if __name__ == "__main__":
    asyncio.run(main())
```

NOTE

A non-blocking pipeline on one thread: the producer fills the queue, the consumer processes items in the background, and q.join() confirms completion.

ASYNC MIXING

Integrating Synchronous Blocking Code

- A blocking call such as time.sleep() inside a coroutine freezes the whole event loop.
- asyncio.to_thread() runs the blocking function in a separate worker thread.
- The event loop stays free to run other coroutines in the meantime.
- Useful for older synchronous libraries and file operations that cannot be awaited.

```python
import asyncio
import time

def blocking_write(data):
    # Simulate a blocking synchronous disk write
    print(f"Starting write for {data}...")
    time.sleep(0.5)
    print(f"Finished write for {data}")
    return f"Saved {data}"

async def main():
    # Offload the blocking function safely
    result = await asyncio.to_thread(
        blocking_write, "dataset_v6"
    )
    print(f"Outcome: {result}")

if __name__ == "__main__":
    asyncio.run(main())
```

NOTE

asyncio.to_thread() moves the slow blocking_write() into a worker thread, so the event loop never stalls.

SUMMARY

Choosing the Right Approach

Approach	How it runs	Best for	Python tools
Synchronous	One task at a time, in order	Simple scripts and short tasks	Plain functions
Multithreading	Several OS threads; the GIL lets one run Python at a time	I/O-bound work using blocking libraries	threading, queue.Queue
asyncio	One thread; coroutines switch only at await	Many concurrent I/O tasks (network, APIs)	asyncio.gather, asyncio.Queue
Multiprocessing	Separate processes, each with its own interpreter and GIL	CPU-bound work (calculation, image processing)	multiprocessing, concurrent.futures

NOTE

Rule of thumb: if the program mostly waits (network, disk, database), use asyncio or threads; if it mostly computes, use separate processes, or a free-threaded Python build once your libraries support it.

ACTIVITIES

8 Activities to Consolidate Concepts

- Activity 1: Simple Async Greeting. Declare and await a basic coroutine that prints a greeting.
- Activity 2: Concurrent Task Scheduling. Schedule several API request checkers to run concurrently.
- Activity 3: Bulk Gathering of Awaitables. Run a list of scraper coroutines with asyncio.gather.
- Activity 4: Bounded Web Requests with Timeouts. Use asyncio.wait_for so the event loop never hangs.
- Activity 5: State Synchronisation with Locks. Remove race hazards on shared writes with asyncio.Lock.
- Activity 6: Throttling Concurrency with Semaphores. Limit parallel API requests to 2 at a time.
- Activity 7: Offloading Blocking Synchronous Tasks. Run heavy blocking work in worker threads without stalling the loop.
- Activity 8: Producer-Consumer Pipeline. Build an async ETL pipeline with asyncio.Queue and Semaphores.

```
# Verification Guidelines:
# 1. Run templates from the outbox.
# 2. Complete the # TODO placeholders.
# 3. Test normal completion and error states.
# 4. Verify thread and asyncio behaviours.
# 5. Review official solutions when done.
```